

# SkillsTrack Learner Support Portal

*JavaScript Programmer – Two-Month Integrated Project*

## Month 1 – Formative Question Answer Sheet (92 Marks)

### Project Details

**Learner name:** Chad Teixeira

**Team name:** BMC

**Assessor:**

**Project start date:** 2 August 2026

**Final due date:** 2 September 2026

---

### Month 1 Formative Questions (92 Marks)

1. Explain why the programming life cycle should be followed before coding begins. (2 marks)

The programming life cycle acts as a set of steps you follow before and during developing and building software. These steps are analysis, design, coding, testing, documentation, and maintenance. Following these steps before coding makes sure that you understand the problem properly and have a plan, so you don't waste time building the wrong thing. It also means fewer mistakes, and makes the finished program easier to test, fix, maintain, and understand.

---

2. List and explain the main steps of the programming life cycle. Your answer must show how each step contributes to the development of a working solution. (15 marks)

**Problem Analysis:** Talk with the client to find out exactly what the program needs to do like its inputs, the processing, and the outputs. This makes sure that you are solving the right before any design or coding starts.

**Design:** Plan the solution on paper before coding, using pseudocode, flowcharts, diagrams, and screen sketches. This gives a clear plan to follow and lets mistakes get caught early; while they are still cheap to fix.

**Coding / Implementation:** Turn the design into a real program using code. This is the step that actually builds the working software. Good design makes this step faster and may produce fewer bugs

**Testing:** Run the program against test cases to check that it works correctly and matches requirements. This finds bugs and or gaps before the program reaches real users.

**Documentation:** Write notes for other developers and guides for users. This helps others maintain the program later as well as helps users to use the program correctly.

Debugging: Trace the bugs found in testing back to their cause in the code, and fix them. This makes the program more reliable and accurate.

Implementation / Deployment: install the finished, tested program so users can start using it. This is the step where the program starts being useful.

Maintenance: After release, you should keep monitoring and updating the program as well as fixing new bugs and adding features as needs change. This keeps the software useful and up to date

---

### 3. Explain when const should be used instead of let. Also explain why var should normally be avoided in modern JavaScript. (5 marks)

You should use const when a variable's value will not need to change after it is first set. Example: a fixed setting, or a value that is calculated once and never changed again. Use let when the value is expected to change while the program runs. Example: a counter in a loop, or a total that keeps getting updated.

Var should be avoided in modern JavaScript for a few reasons:

It allows the same variable name to be declared again in the same scope without an error, so it's easy to accidentally overwrite a variable.

It is function-scoped, not block-scoped — a var declared inside an if statement or loop can 'leak' out and stay accessible outside that block, which causes confusing bugs.

It is hoisted and starts as undefined, so it can be used before its declaration line runs — this can hide mistakes that let and const would immediately flag as an error.

---

### 4. Explain how local and global scope can affect the reliability and maintainability of a JavaScript application. (2 marks)

A local variable only exists inside the function or block where it was declared, so no other part of the program can accidentally read or change it. This makes the program more reliable. A global variable can be accessed and be changed from anywhere in the program, which increases the chance of naming clashes and unexpected side effects, since any function could change its value. This makes bugs harder to trace and the code harder to maintain as the program grows. Because of this, it's best to keep variables local wherever possible and use global variables sparingly.

---

5. Explain how map(), filter() and reduce() process an array of task objects differently. Provide one suitable use for each method. (6 marks)

map(): Changes every item in the array and returns a new array of the same length. Example: get just the titles of every task: `tasks.map(task => task.title)`.

filter(): Tests every item and keeps only the ones that pass, returning a new array that may be shorter than the original. Example: get only the completed tasks: `tasks.filter(task => task.completed)`.

reduce(): Combines every item down into one single value. Example: add up the total hours: `tasks.reduce((sum, task) => sum + task.hours, 0)`

---

6. Explain why an application should use classes or structured objects instead of storing related information in several unrelated variables. (5 marks)

Storing related data together in a class or object keeps it organised in one place, which is much easier to read and maintain than lots of separate, unrelated variables. A class is also a reusable template that bundles data and behaviour together, so you can create many objects from the same definition without repeating code. This lowers the risk of data becoming mismatched or forgotten, like updating a name in one place but forgetting a related variable elsewhere, and it makes the program easier to scale, since you can just create a new object instead of declaring more and more variables. It also matches how we think about real-world things, which makes the code easier for other developers to follow.

7. Explain how branches, pull requests and automated checks reduce risk when developers collaborate on a project. (5 marks)

Branches: Let a developer work on a new feature or fix in an isolated copy of the code, without affecting the main code or other people's work, until it's ready to share.

Pull requests: A formal way to propose changes and have them reviewed by teammates before they're merged into the main branch. This catches mistakes early and keeps a record of what changed and why.

Automated checks: Tools like automated tests and linters, run through continuous Integration that automatically check new code before it can be merged, catching bugs early.

---

8. Study the following values: (8 marks total)

```
const userName = "Lerato";
```

```
const age = 22;
```

```
const isActive = true;
```

```
const selectedProject = null;
```

a. State the data type of each value. (4 marks)

---

userName = "Lerato" → String

age = 22 → Number

isActive = true → Boolean

selectedProject = null → Object

---

b. Explain how age could be converted from a number to a string. (1 mark)

wrap the age variable in a string. Example: String(age)

---

c. Explain what the typeof operator is used for. (1 mark)

typeof checks and returns the data type of a value, as a string such as "string", "number", "boolean", "object", "undefined" or "function".

---

9. Analyse the following code: (8 marks total)

```
const total = "10" + 5;
```

```
const looseComparison = 5 == "5";
```

```
const strictComparison = 5 === "5";
```

```
console.log(total);
```

```
console.log(looseComparison);
```

```
console.log(strictComparison);
```

a. State the output of each console.log() statement. (3 marks)

1. "105"

2. True

3. false

---

b. Explain why "10" + 5 does not produce the number 15. (2 marks)

When you use + and one side is a string, JavaScript joins them as text instead of adding them as numbers. It turns the 5 into "5" and sticks it onto "10", giving the string "105" instead of the number 15.

---

c. Explain the difference between == and ===. (2 marks)

== is loose equality which converts both sides to the same type before comparing, so 5 == "5" is true. === is strict equality which compares both the value and the type, with no conversion, so 5 === "5" is false.

---

d. Rewrite the first statement so that it produces the number 15. (1 mark)

```
const total = Number("10") + 5
```

---

10. Study the following functions: (8 marks total)

```
function calculateTotal(price, quantity = 1) {  
  return price * quantity;  
}
```

```
const applyDiscount = (total, percentage) => {  
  return total - total * (percentage / 100);  
};
```

```
const orderTotal = calculateTotal(150, 3);  
const finalTotal = applyDiscount(orderTotal, 10);
```

a. Identify the parameters of calculateTotal(). (2 marks)

The parameters are price and quantity

---

b. Explain the purpose of the default value assigned to quantity. (1 mark)

If calculateTotal() is called without a value for quantity, it automatically uses 1 instead, so the function still works properly rather than returning NaN or undefined.

c. Explain what the return keyword does. (1 mark)

return sends a value back out of the function to wherever it was called from, and immediately stops the function running.

d. State the value stored in orderTotal. (1 mark)

`orderTotal = calculateTotal(150, 3) = 150 × 3 = 450`

---

e. State the value stored in finalTotal. (1 mark)

`finalTotal = applyDiscount(450, 10) = 450 - (450 × 10 / 100) = 450 - 45 = 405.`

---

f. Explain one difference between a function declaration and an arrow function. (2 marks)

One difference is that a function declaration is hoisted, meaning it can be called before the line it's defined on. An arrow function assigned to a const or let is not hoisted the same way, so it must be defined before it's called.

---

11. Study the following array: (10 marks total)

```
const tasks = [  
  { title: "Create wireframes", completed: true, hours: 3 },  
  { title: "Develop login form", completed: false, hours: 5 },  
  { title: "Test application", completed: true, hours: 2 },  
  { title: "Write documentation", completed: false, hours: 4 }  
];
```

Write JavaScript code that:

a. Uses a loop to display the title of every task. (2 marks)

```
for (const task of tasks) {  
  
  console.log(task.title);  
  
}
```

---

b. Uses a conditional statement to display only completed tasks. (2 marks)

```
for (const task of tasks) {  
  
  if (task.completed)  
  {  
    console.log(task.title);  
  }  
  
}
```

---

c. Counts the number of completed tasks. (2 marks)

```
const completedCount = tasks.filter(task => task.completed).length;
```

---

d. Calculates the total number of hours for all tasks. (2 marks)

```
const totalHours = tasks.reduce((sum, task) => sum + task.hours, 0)
```

---

e. Displays an appropriate message if no tasks are available. (2 marks)

```
if (tasks.length === 0) {  
  console.log("No tasks are available.");  
} else {  
  // display or process the tasks as normal  
}
```

---

15. Study the following statement: (4 marks total)

```
document.cookie = "theme=dark; max-age=3600; path=/";
```

a. Explain what information the cookie stores. (1 mark)

it stores a cookie called theme with the value dark. a small piece of data is saved in the users browser, in this case remembering that the user chose the dark theme, so it can be recalled next time

---

b. Explain the purpose of max-age=3600. (1 mark)

It sets how long the cookie lasts, in seconds. The cookie automatically expires and is deleted 3600 seconds (1 hour) after it was created.

---

c. Explain the purpose of path=/. (1 mark)

It says which part of the site the cookie can be used on. path=/ means the cookie is available on every page of the site, not just one folder or page.

---

d. Write a JavaScript statement that displays the available cookies. (1 mark)

```
console.log(document.cookie);
```

---

## Testing and Problem-Solving

### 16. A registration form contains the following fields: name; email address; password; and age. (15 marks)

Develop five test cases that could be used to test the form. Each test case must include the input or condition being tested and the expected result. Your test cases must include at least one valid submission, one missing value, one invalid email address and one boundary-value test.

---

| Test Case: Input / Condition Tested                                                                                | Expected Result                                                                                                                                                       |
|--------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Name: "Chad Teixeira", Email: "Chad@example.com", Password: "Secure@123", Age: 20 (all fields filled in correctly) | The form is accepted: no error messages are shown, and the data is saved successfully.                                                                                |
| Name field left blank; Email, Password and Age all filled in correctly                                             | The form does not submit. An error such as "Name is required" is shown, and the user is asked to complete the field.                                                  |
| Email entered as "thabo.example.com" (missing the @ symbol); other fields valid                                    | The form does not submit. An error such as "Please enter a valid email address" is shown.                                                                             |
| Age entered as 17, one year below the minimum allowed age of 18; other fields valid                                | The form does not submit. An error such as "You must be at least 18 years old to register" is shown. (At the boundary itself, Age = 18, the form should be accepted.) |
| Password entered as "123" (under the required minimum of 8 characters); other fields valid                         | The form does not submit. An error such as "Password must be at least 8 characters long" is shown.                                                                    |

---



## Academic Integrity and Authenticity Declaration

I declare that:

- the evidence submitted for assessment accurately represents my own contribution;
- I have acknowledged tutorials, libraries, code examples, artificial-intelligence assistance and other external sources;
- I understand the JavaScript, Firebase and GitHub work that I am presenting;
- I can explain, test and modify the code attributed to me;
- I have not stored passwords, private credentials or confidential data in the repository;
- I understand that if I cannot demonstrate understanding OR provide enough evidence, my project could not be accepted and therefore I will not pass the FISA (Final Integrated Supervised Assessment).

**Learner name:** Chad Teixeira

**Learner signature:** C.T

**Date:** 31/08/2026

---

**Assessor name:**

**Assessor signature:**

**Date:**

---